

Catálogo de Star Wars — Backend

Backend desarrollado en **NestJS** para la gestión de películas, con autenticación JWT, control de roles, y sincronización de datos desde la API pública de Star Wars (SWAPI).

Proyecto realizado como parte del **Backend Nest Ssr. Test** de Conexa.

Stack

- **Framework:** NestJS
- **Base de datos:** PostgreSQL (hosteada en Railway)
- **ORM:** Prisma
- **Autenticación:** JWT (@nestjs/jwt, passport-jwt)
- **Hashing de contraseñas:** bcrypt
- **Documentación de API:** Swagger (@nestjs/swagger)
- **Testing:** Jest

Requisitos previos

- **Node.js** (v18 o superior recomendado). Si no lo tenés instalado, descargalo desde nodejs.org (la versión LTS es la recomendada). npm se instala automáticamente junto con Node.
- Una instancia de PostgreSQL (local vía Docker, o un servicio hosteado como Railway/Supabase)

Para verificar que Node y npm están instalados correctamente:

```
node -v
npm -v
```

Instalación

Cloná el repositorio e instalá las dependencias:

```
git clone <URL_DEL_REPO>
cd star-wars-catalogue
npm install
```

Variables de entorno

Creá un archivo `.env` en la raíz del proyecto con las siguientes variables:

```
DATABASE_URL="postgres://usuario:password@host:puerto/nombre_db"
JWT_SECRET="una_clave_secreta_larga_y_aleatoria"
```

⚠ `JWT_SECRET` es la clave usada para firmar y verificar los tokens de sesión. No debe compartirse ni subirse al repositorio.

Un archivo `.env.example` está incluido como plantilla de referencia.

Migraciones de base de datos

Con `DATABASE_URL` ya configurada, aplicá las migraciones para crear las tablas:

```
npx prisma migrate dev
```

Esto también genera el cliente de Prisma automáticamente. Si necesitas regenerarlo manualmente en algún momento:

```
npx prisma generate
```

Levantar el proyecto

```
npm run start:dev
```

La aplicación corre por defecto en `http://localhost:3000`.

Documentación de la API (Swagger)

Con el proyecto corriendo, la documentación interactiva está disponible en:

`http://localhost:3000/docs`

Desde ahí se pueden probar todos los endpoints directamente. Para los endpoints protegidos, hacé clic en **Authorize** (arriba a la derecha) y pegá el token obtenido en `/auth/login`.

Autenticación y roles

- **Registro:** `POST /auth/registrarse` — crea un usuario con contraseña hasheada. Todo usuario nuevo se crea con rol `REGULAR`.
- **Login:** `POST /auth/login` — devuelve un `access_token` (JWT) si las credenciales son correctas.
- **Roles:** `REGULAR` y `ADMINISTRADOR`. Los endpoints de películas están protegidos según el rol correspondiente (ver Swagger para el detalle de cada uno).

Para probar los endpoints de administrador, hay que promover manualmente un usuario a `ADMINISTRADOR` (por ejemplo, con `npx prisma studio`), ya que no existe un endpoint público para autoasignarse ese rol.

Sincronización con SWAPI

El endpoint `POST /peliculas/sync` (solo administradores) trae los datos de las películas desde SWAPI y los guarda en la base de datos. Usa `upsert` sobre el campo `swapiId`, por lo que puede ejecutarse múltiples veces sin generar duplicados.

Correr los tests

```
npm run test
```

Esto corre la suite completa de tests unitarios, cubriendo: - Lógica de autenticación (registro, login, hashing, validación de credenciales) - CRUD y búsqueda de películas - Guard de roles - Mapeo y sincronización de datos de SWAPI

Estructura del proyecto

```
src/  
  auth/           # Registro, login, JWT, guards y decoradores de roles  
  peliculas/      # CRUD de películas, búsqueda, sincronización con SWAPI  
  swapi/          # Cliente y mapper de la API de Star Wars  
  prisma/         # Servicio y módulo de Prisma (conexión a la base de datos)  
  common/        # Decoradores compartidos (@Allow, @Roles)
```

```
prisma/  
  schema.prisma    # Modelos de datos (Usuario, Pelicula)  
  migrations/      # Historial de migraciones
```

Notas de diseño

- El `id` de cada película es un UUID generado internamente, independiente del `id` que provee SWAPI. El campo `swapiId` guarda ese identificador externo solo cuando la película proviene de una sincronización.
- Las contraseñas nunca se almacenan ni se devuelven en texto plano — se hashean con `bcrypt` antes de guardarse, y se excluyen de toda respuesta de la API.
- Los mensajes de error de login son intencionalmente genéricos (mismo mensaje para “usuario no existe” y “contraseña incorrecta”) para no revelar qué emails están registrados.